Comprehensive DSA Learning Summary

LeetCode Problem Solving Journey

Rahul Sunda

Roll No: 24B2147

Data Structures and Algorithms

Topics Covered:

Binary Search	**Linked Lists**	**Stacks & Queues**
Arrays	**Strings**	**Backtracking**
Combinations	**Matrices**	**Dynamic Programming**

June 25, 2026

Contents

List of Figures

List of Tables

1 Introduction

This document provides a comprehensive summary of the Data Structures and Algorithms concepts learned through solving LeetCode problems across multiple weeks. The journey covered over 50 problems, ranging from easy to hard difficulty, across various topics.

2 Binary Search Mastery

2.1 Standard Binary Search Fundamentals

2.1.1 704. Binary Search (Easy)

- Learned the basic implementation with left and right pointers
- Key Insight: `mid = left + (right - left)/2` prevents integer overflow

```
int binarySearch(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while(left <= right) {
        int mid = left + (right - left) / 2;
        if(nums[mid] == target) return mid;
        else if(nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Listing 1: Basic Binary Search Template

2.1.2 35. Search Insert Position (Easy)

- Found where to insert a value maintaining sorted order
- Key Insight: When not found, `left` gives the insertion position

2.1.3 34. Find First and Last Position of Element in Sorted Array (Medium)

- Running binary search twice - one for left boundary, one for right boundary
- First search: `if(nums[mid] >= target) right = mid - 1` to find first occurrence
- Second search: `if(nums[mid] <= target) left = mid + 1` to find last occurrence

2.2 Rotated Array Binary Search

Key Concept: At least one half is always sorted in a rotated sorted array.

```cpp
int search(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while(left <= right) {
        int mid = left + (right - left) / 2;
        if(nums[mid] == target) return mid;

        if(nums[left] <= nums[mid]) {  // Left half is sorted
            if(target >= nums[left] && target < nums[mid])
                right = mid - 1;
            else
                left = mid + 1;
        } else {  // Right half is sorted
            if(target > nums[mid] && target <= nums[right])
                left = mid + 1;
            else
                right = mid - 1;
        }
    }
    return -1;
}
```

Listing 2: Search in Rotated Sorted Array

Table 1: Rotated Array Problems Solved

ID	Problem	Key Learning
33	Search in Rotated Sorted Array	Identifying which half is sorted
81	Search in Rotated Sorted Array II	Handling duplicates
153	Find Minimum in Rotated Sorted Array	Comparing with rightmost element

2.3 Binary Search on Answer Space

The Paradigm Shift: Instead of searching in the array, search in the range of possible answers.

```cpp
bool feasible(int mid) {
    // Check if answer 'mid' works
    // Return true if possible, false otherwise
}

int binarySearchOnAnswer() {
    int left = min_possible, right = max_possible;
    while(left < right) {
        int mid = left + (right - left) / 2;
        if(feasible(mid)) right = mid;  // Minimize answer
        else left = mid + 1;
    }
    return left;
}
```

Listing 3: Binary Search on Answer Pattern

Table 2: Binary Search on Answer Problems

ID	Problem	Feasibility Function
875	Koko Eating Bananas	Can eat all bananas in H hours
1011	Capacity to Ship Packages	Can ship in D days
1283	Smallest Divisor Given Threshold	Sum of ceil divisions threshold
1482	Minimum Days to Make Bouquets	Can make m bouquets

2.4 Advanced Binary Search Patterns

2.4.1 540. Single Element in a Sorted Array (Medium)

- Elements appear twice except one - find the odd one
- Key Insight: Check parity of mid index to decide which side has the single element

```cpp
int singleNonDuplicate(vector<int>& nums) {
    int left = 0, right = nums.size() - 1;
    while(left < right) {
        int mid = left + (right - left) / 2;
        if(nums[mid] == nums[mid ^ 1])  // mid^1 gives paired index
            left = mid + 1;
        else
            right = mid;
    }
    return nums[left];
}
```

Listing 4: Single Element in Sorted Array

Note on `mid` ^{1} : $XORwith1 flips the last bit, giving the paired index$:

`If mid is even:` `mid` $^{1} = mid + 1$

`If mid is odd:` `mid` $^{1} = mid - 1$

2.4.2 162. Find Peak Element (Medium)

Needs Revision

- Finding local maxima without sorting
- Key Insight: If `nums[mid] < nums[mid+1]`, peak is on right side
- **Note**: This problem needs more practice with edge cases

3 Linked List Mastery

3.1 Fundamental Operations

```cpp
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while(curr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Listing 5: Reverse Linked List (Iterative)

3.1.1 876. Middle of the Linked List (Easy)

- Slow-fast pointer technique (Tortoise and Hare)
- Slow moves 1 step, fast moves 2 steps
- When fast reaches end, slow is at middle

3.2 Cycle Detection

Floyd's Cycle Detection Algorithm:

1. Use slow and fast pointers
2. If they meet, cycle exists
3. Reset slow to head, move both at same speed - they meet at cycle start

```cpp
ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    // Find meeting point
    while(fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast) {
            // Find cycle start
            slow = head;
            while(slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }
    return nullptr;
}
```

Listing 6: Linked List Cycle II

Mathematical Proof:

$$\text{Distance}_{\text{slow}} = X + Y \tag{1}$$
$$\text{Distance}_{\text{fast}} = X + Y + n \cdot C \tag{2}$$
$$\text{fast} = 2 \times \text{slow} \tag{3}$$
$$X = (n-1) \times C + Z \tag{4}$$

3.3 Advanced Linked List Manipulation

Table 3: Advanced Linked List Problems

ID	Problem	Technique
234	Palindrome Linked List	Reverse second half and compare
328	Odd Even Linked List	Group by index parity
19	Remove Nth Node From End	Two-pointer with offset
2095	Delete the Middle Node	Find middle with prev pointer
148	Sort List	Merge sort on linked list

```
ListNode* sortList(ListNode* head) {
    if(!head || !head->next) return head;

    // Find middle
    ListNode* slow = head;
    ListNode* fast = head->next;
    while(fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Split
    ListNode* right = slow->next;
    slow->next = nullptr;

    // Recursively sort
    ListNode* left = sortList(head);
    right = sortList(right);

    // Merge
    return merge(left, right);
}
```

Listing 7: Merge Sort on Linked List

3.3.1 493. Reverse Pairs (Hard)

- Counting inversions with a twist using modified merge sort
- Count pairs before merging to ensure proper ordering

4 Stacks and Queues

4.1 Stack Fundamentals

```cpp
bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> mapping = {
        {')', '('}, {']', '['}, {'}', '{'}
    };

    for(char c : s) {
        if(mapping.count(c)) {
            if(st.empty() || st.top() != mapping[c]) return false;
            st.pop();
        } else {
            st.push(c);
        }
    }
    return st.empty();
}
```

Listing 8: Valid Parentheses

4.1.1 155. Min Stack (Medium)

- Maintaining minimum in O(1)
- Two approaches: two stacks or single stack with pairs

4.2 Next Greater Element Pattern

Monotonic Stack Pattern:

```cpp
vector<int> nextGreaterElement(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, -1);
    stack<int> st;

    for(int i = n-1; i >= 0; i--) {
        while(!st.empty() && st.top() <= nums[i]) st.pop();
        if(!st.empty()) result[i] = st.top();
        st.push(nums[i]);
    }
    return result;
}
```

Listing 9: Next Greater Element

Table 4: Stack Problems

ID	Problem	Pattern
496	Next Greater Element I	Monotonic decreasing stack
503	Next Greater Element II	Circular array with 2n traversal
20	Valid Parentheses	Stack matching
155	Min Stack	Maintain minimum

4.3 Implementation Problems

4.3.1 232. Implement Queue using Stacks (Easy)

- Two-stack approach: input and output stacks
- Each element moves at most once, amortized O(1)

4.3.2 225. Implement Stack using Queues (Easy)

- One queue with rotation: push O(n), pop O(1)

4.3.3 460. LFU Cache (Hard)

- Most complex data structure - Least Frequently Used cache
- Data structures:
  - HashMap for O(1) access
  - Frequency buckets
  - Doubly linked lists

```
struct Node {
    int key, val, freq;
    Node(int k, int v, int f) : key(k), val(v), freq(f) {}
};

class LFUCache {
    int capacity, minFreq;
    unordered_map<int, Node*> keyNode;
    unordered_map<int, list<Node*>> freqList;

    void updateFreq(Node* node) {
        // Remove from current freq list
        // Increment frequency
        // Add to new freq list
        // Update minFreq if needed
    }
};
```

Listing 10: LFU Cache Structure

5 Array Manipulation

5.1 Two-Pointer Techniques

5.1.1 Dutch National Flag Algorithm (75. Sort Colors)

```cpp
void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;
    while(mid <= high) {
        if(nums[mid] == 0) {
            swap(nums[low], nums[mid]);
            low++; mid++;
        } else if(nums[mid] == 1) {
            mid++;
        } else { // nums[mid] == 2
            swap(nums[mid], nums[high]);
            high--;
        }
    }
}
```

Listing 11: Sort Colors

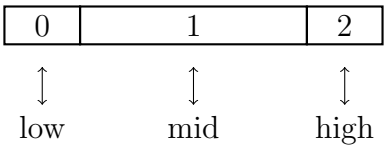

Figure 1: Dutch National Flag Visualization

5.2 Prefix/Suffix Techniques

```cpp
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, 1);

    // Left products
    for(int i = 1; i < n; i++) {
        ans[i] = ans[i-1] * nums[i-1];
    }

    // Right products multiplied
    int right = 1;
    for(int i = n-1; i >= 0; i--) {
        ans[i] *= right;
        right *= nums[i];
    }
    return ans;
}
```

Listing 12: Product of Array Except Self

5.2.1 73. Set Matrix Zeroes (Medium)

- Use first row and first column as markers
- Separate check for first row and first column
- Zero out based on markers

5.3 Majority Element Algorithms

Boyer-Moore Voting Algorithm:

```
int majorityElement(vector<int>& nums) {
    int candidate = 0, count = 0;
    for(int num : nums) {
        if(count == 0) candidate = num;
        if(num == candidate) count++;
        else count--;
    }
    return candidate;
}
```

Listing 13: Majority Element

Table 5: Majority Element Problems

ID	Problem	Algorithm
169	Majority Element	Boyer-Moore (single candidate)
229	Majority Element II	Boyer-Moore (two candidates)

6 Strings and Backtracking

6.1 String Processing

6.1.1 5. Longest Palindromic Substring (Medium)

- Expand around center technique
- Two cases: odd length and even length

```
string longestPalindrome(string s) {
    if(s.empty()) return "";
    int start = 0, maxLen = 1;

    for(int i = 0; i < s.length(); i++) {
        // Odd length
        expand(s, i, i, start, maxLen);
        // Even length
        expand(s, i, i+1, start, maxLen);
    }
```

```
    return s.substr(start, maxLen);
}

void expand(string& s, int left, int right, int& start, int& maxLen) {
    while(left >= 0 && right < s.length() && s[left] == s[right]) {
        left--; right++;
    }
    int len = right - left - 1;
    if(len > maxLen) {
        start = left + 1;
        maxLen = len;
    }
}
```

Listing 14: Longest Palindromic Substring

6.1.2 1781. Sum of Beauty of All Substrings (Medium)

- Beauty = max freq − min freq in substring
- Maintain frequency array for each substring

6.2 Backtracking

```
void backtrack(vector<int>& nums, int start, vector<int>& current,
    vector<vector<int>>& result) {
    // Add current subset
    result.push_back(current);

    for(int i = start; i < nums.size(); i++) {
        // Handle duplicates
        if(i > start && nums[i] == nums[i-1]) continue;

        current.push_back(nums[i]);
        backtrack(nums, i+1, current, result);
        current.pop_back();
    }
}
```

Listing 15: Backtracking Pattern

Table 6: Backtracking Problems Solved

ID	Problem	Special Constraint
78	Subsets	Generate all subsets
90	Subsets II	Handle duplicates
216	Combination Sum III	Exactly k numbers, sum to n
40	Combination Sum II	Each element used once

7 Algorithmic Patterns Mastered

Table 7: Pattern Recognition Guide

Pattern	When to Use	Example Problems
Binary Search	Sorted/partially sorted data	33, 153, 875
Two Pointer	Compare elements, in-place operations	283, 26, 75
Monotonic Stack	Next greater/smaller element	496, 503
Backtracking	Generate all combinations	78, 90, 216
Prefix/Suffix	Need cumulative info	238, 73
Floyd's Cycle	Cycle detection in linked lists	141, 142
Boyer-Moore	Majority element detection	169, 229
Merge Sort	Linked list sorting	148, 493

8 Areas for Improvement

8.1 Concepts Needing Revision

1. **Find Peak Element (162)** - Binary search edge cases
2. **Count Primes (204)** - Sieve implementation details
3. **LFU Cache (460)** - Complex data structure practice

8.2 Topics for Further Study

1. Sliding Window
2. Dynamic Programming (advanced)
3. Trees and Graphs
4. Heap/Priority Queue

9 Complexity Cheat Sheet

Table 8: Time and Space Complexities

Algorithm	**Time Complexity**	**Space Complexity**
Binary Search	O(log n)	O(1)
Two Pointer	O(n)	O(1)
Monotonic Stack	O(n)	O(n)
Backtracking (Subsets)	$O(2^n)$	O(n)
Merge Sort	O(n log n)	O(n)
Floyd's Cycle Detection	O(n)	O(1)
Boyer-Moore Voting	O(n)	O(1)
Dutch National Flag	O(n)	O(1)

10 Conclusion

10.1 Key Takeaways

- Developed strong understanding of Binary Search including complex variations
- Mastered Linked List operations and cycle detection
- Learned Stack patterns (monotonic, parentheses, min-stack)
- Gained proficiency in Backtracking for combinations
- Built foundation for complex data structures (LFU Cache)
- Learned to identify and apply appropriate algorithmic patterns

10.2 Next Steps

1. Practice more Hard problems
2. Cover remaining topics: Trees, Graphs, DP
3. Focus on optimizing existing solutions
4. Participate in coding contests
5. Build personal projects using DSA concepts